

Quem decide o que o sistema precisa fazer

O processo de Engenharia de Requisitos, seus envolvidos, e por que um requisito perfeito levantado com a pessoa errada continua sendo um requisito errado.

O que veremos hoje

BLOCO 1

O processo

As cinco atividades da engenharia de requisitos, os três níveis da norma ISO/IEC/IEEE 29148, e por que essas atividades **não acontecem em fila**.

BLOCO 2

Os envolvidos

O que é uma parte interessada, os papéis do processo, o modelo cebola e o interessado que ninguém lembra: o que quer que o sistema falhe.

BLOCO 3

Priorizar os envolvidos

A matriz poder e interesse, as quatro estratégias de engajamento, a crítica honesta a esse modelo, e o modelo de saliência.

BLOCO 4

O caso

Um programa público de saúde de 9,8 bilhões de libras, desmontado depois de nove anos por ter ignorado quem usaria o sistema.

POR QUE ESTA AULA IMPORTA

Na Aula 1 vocês viram **que** projetos falham por requisitos. Na Aula 2, **como se escreve** um requisito bom. Falta a pergunta que decide as duas coisas: **com quem você fala?**

Um requisito impecável, verificável e sem ambiguidade, levantado com a pessoa errada, continua sendo um requisito errado — e custa exatamente o mesmo para descobrir tarde. Hoje a aula trata do processo que produz o requisito e das pessoas de quem ele vem.

Formato de hoje: aula expositiva, sem prática no computador. O laboratório com o projeto das equipes começa no próximo encontro, dia 10 de setembro.

Por que projetos falham por requisitos

PONTO 1

O defeito nasce antes do código

A maior parte dos defeitos caros de um sistema não é erro de programação. É requisito ausente, ambíguo ou entendido de forma diferente por duas pessoas. O código está correto — ele faz exatamente o que foi pedido.

PONTO 2

O custo cresce a cada fase

Corrigir um requisito enquanto ele é uma frase no documento custa quase nada. Corrigir depois de codificado custa caro. Corrigir com o sistema em produção, com dado real e usuário dentro, custa ordens de grandeza mais.

PONTO 3

A dinâmica do pedido vago

A turma recebeu uma frase ambígua e construiu produtos diferentes a partir dela. Ninguém errou. A frase é que admitia várias leituras — e ninguém percebeu isso até comparar os resultados.

A conclusão da Aula 1: **o momento mais barato de consertar um sistema é antes de ele existir.** É exatamente por isso que a engenharia de requisitos é uma disciplina, e não uma conversa informal antes de começar a programar.

A relação entre a fase em que o defeito é encontrado e o custo de corrigi-lo foi formalizada por Barry Boehm em Software Engineering Economics (Prentice-Hall, 1981) e é reaplicada em praticamente toda a literatura da área desde então.

O que caracteriza um requisito

Três coisas diferentes que costumam ser chamadas pelo mesmo nome:

TIPO	O QUE É	EXEMPLO
Requisito funcional	O que o sistema deve fazer	O sistema deve emitir a segunda via do boleto
Requisito não funcional	Sob que qualidade ele deve fazer	A segunda via deve ser emitida em até 3 segundos
Regra de negócio	A política da organização, que existe mesmo sem sistema	Segunda via só é emitida para boleto vencido há menos de 30 dias

E as nove características que a norma ISO/IEC/IEEE 29148 exige de cada requisito individual:

necessário	apropriado	não ambíguo	completo	singular	viável	verificável	correto	conforme
------------	------------	-------------	----------	----------	--------	-------------	---------	----------

E o formato de critério de aceite que a turma treinou: **Dado** que o contexto é este, **Quando** isso acontece, **Então** o sistema faz aquilo. É o que transforma "o sistema deve ser rápido" em algo que alguém consegue testar.

A Aula 2 ensinou a **julgar um requisito já escrito**. Ela não respondeu de onde ele veio, nem quem tinha autoridade para pedi-lo. É esse buraco que a aula de hoje fecha.

Três perguntas, e a que ainda está aberta

As três primeiras aulas da disciplina não são assuntos separados. São três perguntas encadeadas sobre o mesmo problema.

AULA 1 · RESPONDIDO

Por que isso importa?

Projetos falham por requisitos, e o defeito nasce antes da primeira linha de código. Quanto mais tarde ele é descoberto, mais caro fica.

AULA 2 · RESPONDIDO

Como se escreve um requisito bom?

Funcional, não funcional ou regra de negócio; as nove características da norma; critério de aceite testável em Dado, Quando, Então.

HOJE · EM ABERTO

De quem vem o requisito, e quem decide?

Qual é o processo que produz o requisito, quem são as pessoas envolvidas nele, e o que fazer quando elas discordam entre si.

Um requisito impecável levantado com a pessoa errada é **um requisito errado bem escrito**. A qualidade da redação não corrige a origem — e é a origem que a aula de hoje trata.

Por que isso vale para a carreira de vocês: o analista que só escreve bem é substituível. O que sabe **com quem falar**, em que ordem, e como decidir quando dois interessados querem coisas opostas, é o que sustenta um projeto inteiro.

PERGUNTA DE ABERTURA

O diretor pediu. O analista escreveu certo. O funcionário que usa todo dia detestou. Quem estava errado?

Nenhum dos três, individualmente. O erro está antes: alguém decidiu que ouvir o diretor era suficiente. T

BLOCO 1

O processo

As cinco atividades da engenharia de requisitos, os três níveis de requisito da norma, e a razão pela qual esse processo é uma espiral, e não uma fila de etapas que terminam.

O que é Engenharia de Requisitos

O processo de **descobrir, analisar, documentar e verificar** os serviços que um sistema deve prestar e as restrições sob as quais ele deve operar.

Repare em duas palavras da definição. **Descobrir** — não "receber": o requisito raramente chega pronto, ele é construído junto com quem tem o problema. E **verificar** — a engenharia de requisitos não termina quando o documento fica pronto; ela termina quando alguém confirma que o documento descreve a necessidade real.

Os termos, em inglês e em português

requirements engineering — engenharia de requisitos

elicitation — elicitación, ou levantamento

specification — especificação

validation — validação

requirements management — gerência de requisitos

Duas perguntas que não são a mesma

Verificação: estamos construindo o sistema **do jeito certo**? Compara o produto com a especificação.

Validação: estamos construindo **o sistema certo**? Compara a especificação com a necessidade real.

- Um sistema pode passar em toda a verificação e reprovar na validação. Foi o que aconteceu no caso do Bloco 4.

KOTONYA, G.; SOMMERVILLE, I. Requirements Engineering: Processes and Techniques. Wiley, 1998.
SOMMERVILLE, I. Engenharia de Software. 10. ed., capítulo 4. · ISO/IEC/IEEE 29148:2018.

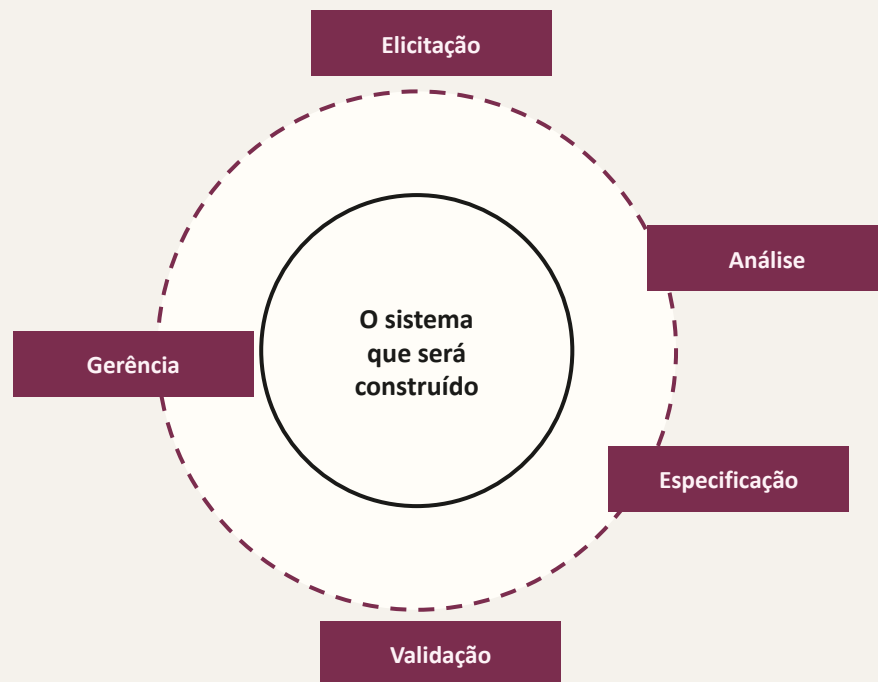
As cinco atividades do processo

ATIVIDADE	O QUE ACONTECE	O QUE SAI DAQUI	O ERRO TÍPICO
1. Elicitação (elicitation)	Descobrir, com as pessoas envolvidas, o que o sistema precisa fazer e por quê	Anotações, transcrições, lista bruta de necessidades	Perguntar só para quem paga
2. Análise	Organizar, classificar, achar conflito entre interessados, negociar e priorizar	Lista consolidada, com conflitos resolvidos e prioridade definida	Aceitar tudo e prometer tudo
3. Especificação	Escrever de forma não ambígua e verificável, no nível certo	O documento de requisitos, com critérios de aceite	Escrever bonito o que foi mal levantado
4. Validação	Confirmar com quem pediu que o escrito descreve a necessidade real	Documento aprovado, ou lista de correções	Validar com quem não vai usar
5. Gerência (management)	Controlar mudança: linha de base, análise de impacto, rastreabilidade	Versões, decisões registradas, matriz de rastreabilidade	Tratar mudança como traição do cliente

As quatro primeiras produzem o requisito. **A quinta cuida dele pelo resto da vida do sistema** — e é a que mais gente esquece que existe, porque o documento parece pronto quando o projeto começa.

Por que isso é uma espiral, e não uma fila

A representação em lista sugere que uma atividade termina para a próxima começar. Não é o que acontece — e tratar assim é a origem de boa parte dos problemas.



Você só entende a necessidade escrevendo-a. Ao especificar, aparecem perguntas que a entrevista não levantou — e é preciso voltar a elicitar.

A validação sempre devolve trabalho. Quando o usuário lê o que foi escrito, ele descobre o que não sabia que queria. Isso não é retrabalho: é o processo funcionando.

O mundo não para durante o projeto. Muda a lei, muda o concorrente, muda o organograma. Requisito é um alvo em movimento, e a gerência existe por causa disso.

Cada volta custa menos que a anterior. Não porque o processo é ineficiente, mas porque cada volta reduz a incerteza. Uma espiral que fecha é o sinal de que os requisitos estabilizaram.

Documento de requisitos não é um portão que se atravessa uma vez. É um artefato vivo, e a linha de base existe justamente para tornar a mudança **visível e negociada**, em vez de proibida.

Os três níveis de requisito da norma

A ISO/IEC/IEEE 29148 separa três documentos, porque três públicos diferentes precisam ler coisas diferentes sobre o mesmo sistema.

NÍVEL	SIGLA NA NORMA	RESPONDE A QUEM	O MESMO REQUISITO, NOS TRÊS NÍVEIS
Requisitos de parte interessada	StRS — Stakeholder Requirements Specification	O interessado, na linguagem dele	A secretaria precisa parar de conferir matrícula uma a uma no papel
Requisitos de sistema	SyRS — System Requirements Specification	O sistema completo: software, hardware, pessoas e processo	O sistema deve validar automaticamente a documentação enviada pelo candidato
Requisitos de software	SRS — Software Requirements Specification	A equipe que vai construir	O módulo deve rejeitar arquivo acima de 5 MB e exibir a mensagem prevista em até 2 segundos

Por que isso importa na prática: confundir os níveis é o erro mais comum de quem começa. Levar ao usuário um requisito escrito no nível de software produz uma reunião em que ninguém entende nada; levar ao programador um requisito escrito no nível do interessado produz um sistema inventado por quem não conhece o negócio.

E a rastreabilidade nasce daqui: cada requisito de software deve poder ser ligado, para cima, à necessidade de um interessado identificado. Requisito que não sobe até ninguém é requisito que alguém inventou.

ISO/IEC/IEEE 29148:2018 — Systems and software engineering: Life cycle processes, requirements engineering.

Três coisas que a engenharia de requisitos não é

NÃO É ISTO

Não é desenhar tela

Protótipo é uma técnica de elicitação e de validação — uma pergunta feita em forma de imagem. Quando a tela vira o requisito, ninguém mais sabe **por que** aquele campo existe, e a primeira mudança de layout apaga a regra junto.

NEM ISTO

Não é um acordo comercial

O documento não existe para proteger o fornecedor do cliente, nem o contrário. Quando ele é escrito para ganhar discussão futura, fica ambíguo de propósito — e ambiguidade proposital é exatamente o defeito que a Aula 2 ensinou a caçar.

MUITO MENOS ISTO

Não é um documento morto

Requisito congelado no primeiro mês descreve um sistema que ninguém mais quer no oitavo. A alternativa não é abandonar o documento: é versioná-lo, com linha de base e análise de impacto — o assunto da Aula 13.

O que ela é, então: **a disciplina de descobrir e registrar, de forma verificável, o que um conjunto de pessoas com interesses diferentes precisa que o sistema faça** — e de manter esse registro honesto enquanto o projeto anda.

Essa frase tem uma expressão que ainda não foi definida nesta disciplina: "**um conjunto de pessoas com interesses diferentes**". É o assunto do próximo bloco, e é onde a maioria dos projetos se perde.

O cliente mudou de ideia na quinta entrega. Isso é falha do cliente, do analista, ou do processo?

Antes de responder, uma distinção: mudou de ideia, ou **descobriu** algo que não sabia quando foi entrevistado? As duas coisas parecem iguais na reunião e são completamente diferentes na análise.

E a pergunta que vem depois: se a mudança fosse inevitável, o que o processo deveria ter feito para que ela chegasse na segunda entrega, e não na quinta?

O que sustenta o processo

01 Descobrir, não receber

O requisito é construído junto com quem tem o problema. Ele quase nunca chega pronto.

02 Cinco atividades

Elicitação, análise, especificação, validação e gerência. As quatro primeiras produzem; a quinta mantém.

03 Espiral, não fila

Especificar levanta perguntas novas, validar devolve trabalho, e o mundo muda durante o projeto.

04 Três níveis

Interessado, sistema e software. Cada requisito de software precisa subir até a necessidade de alguém com nome.

Verificação pergunta se construímos **do jeito certo**. Validação pergunta se construímos **a coisa certa**. Só a segunda depende de ter falado com as pessoas certas.

BLOCO 2

Os envolvidos

O que é uma parte interessada, quais são os papéis do processo, o modelo cebola de Alexander e o interessado que ninguém coloca na lista: aquele que quer que o sistema falhe.

Parte interessada (*stakeholder*)

Qualquer **grupo ou indivíduo que pode afetar, ou ser afetado por**, o alcance dos objetivos da organização. Aplicado ao nosso caso: qualquer pessoa ou grupo afetado pelo sistema, ou capaz de afetá-lo.

Em português a palavra aparece traduzida de três formas — **parte interessada**, **envolvido** e **interessado** — e as três significam a mesma coisa. A norma brasileira usa "parte interessada"; o mercado usa o termo em inglês.

Repare no que a definição inclui

Quem **usa** o sistema, todos os dias
Quem **paga** por ele, e quem aprova o orçamento
Quem **mantém** e opera depois de pronto
Quem é **afetado** sem nunca abrir o sistema — o cidadão que recebe a carta que ele emite
Quem **regula** aquilo: lei, órgão de controle, auditoria

E no que ela não diz

Não diz que todos têm o **mesmo peso** — é o assunto do Bloco 3
Não diz que todos **querem a mesma coisa** — quase nunca querem
Não diz que todos **querem o sucesso** do sistema — alguns preferem que ele falhe, e isso também é informação de requisito

FREEMAN, R. E. Strategic Management: A Stakeholder Approach. Pitman, 1984 — a definição clássica.
ISO/IEC/IEEE 29148:2018 e BABOK v3 trazem a definição operacional usada em projetos de software.

Os papéis do processo e o que se perde ao ignorar cada um

PAPEL	O QUE ELE TRAZ PARA O PROCESSO	O QUE ACONTECE SE ELE FICAR DE FORA
Analista de requisitos	Conduz o processo, traduz entre linguagens e registra a decisão	Cada área especifica sozinha, e ninguém percebe os conflitos
Patrocinador (sponsor)	Autoridade, orçamento e a definição do que é sucesso	O projeto não tem critério de decisão quando há conflito
Usuário final	Como o trabalho realmente acontece, incluindo as exceções do dia a dia	Sistema correto no papel que ninguém consegue usar na prática
Product owner	Prioriza e decide o que entra em cada entrega, no lugar do cliente	A equipe prioriza por conta própria, geralmente pelo que é mais fácil
Especialista de domínio	A regra que ninguém escreveu, e o motivo dela existir	Regra de negócio inventada pela equipe, descoberta em produção
Equipe de operação e suporte	O que quebra às três da manhã e quem é chamado	Sistema sem log, sem monitoramento e impossível de socorrer
Regulador ou auditoria	A exigência que não é negociável e não depende de acordo	Retrabalho caro, ou o sistema barrado antes de entrar no ar

Note a coluna da direita. Nenhuma dessas falhas aparece como erro de programação. Todas aparecem como "o sistema não serve" — meses depois, quando corrigir é caro.

"O cliente" não é uma pessoa só

Quando alguém diz "foi o cliente que pediu", vale perguntar qual deles. Em qualquer organização, três papéis diferentes se escondem sob a mesma palavra — e eles raramente querem a mesma coisa.

COMPRADOR

Quem paga

Quer previsibilidade, custo controlado e um indicador que possa mostrar para a diretoria.

PATROCINADOR

Quem decide

Quer que o projeto atenda à política da área e não gere conflito com os pares dele.

USUÁRIO

Quem usa

Quer terminar a tarefa em menos cliques do que hoje, sem perder o atalho que ele já tem.

Classes de usuário (*user classes*)

Nem todo usuário é igual. Wiegers propõe separar os usuários em classes por frequência de uso, privilégio, domínio da ferramenta e tarefa executada — e tratar cada classe como uma fonte distinta de requisitos. Atendente, supervisor e auditor usam o mesmo sistema e precisam de coisas diferentes dele.

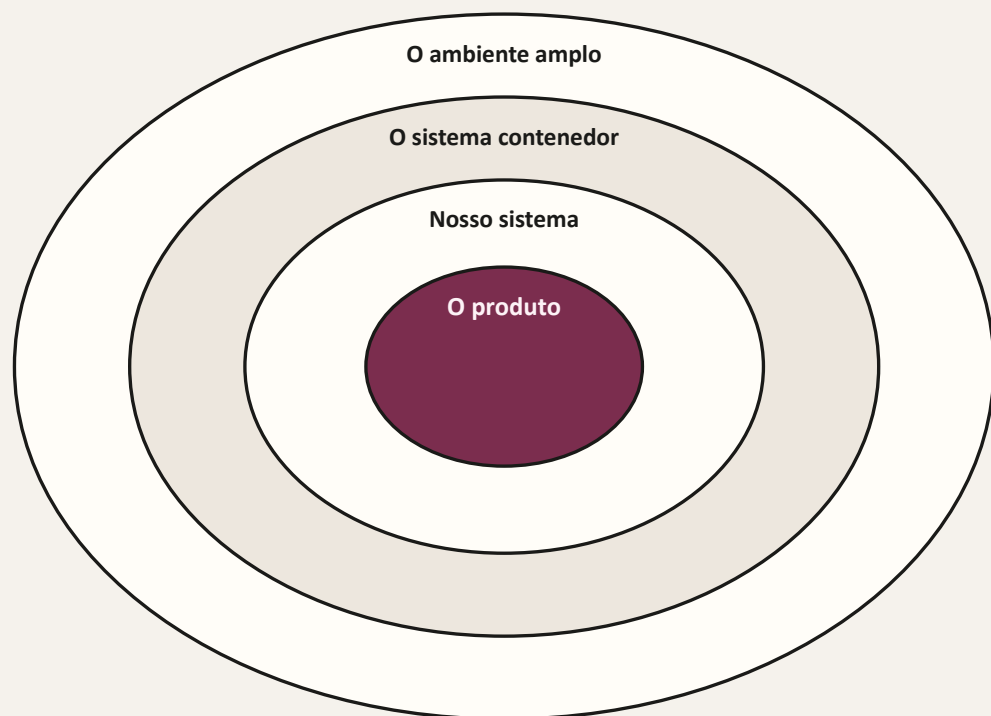
Campeão do produto (*product champion*)

Para cada classe de usuário, um representante nomeado, com tempo alocado e autoridade para falar pela classe. Não é "alguém que a gente entrevistou uma vez": é um papel formal, com nome no documento. Sem isso, a classe silenciosa vira a classe insatisfeita.

WIEGERS, K.; BEATTY, J. Software Requirements. 3. ed. Microsoft Press, 2013 — capítulo sobre encontrar a voz do usuário.

O modelo cebola (*onion model*)

Um mapa para não esquecer ninguém. Cada anel é uma distância em relação ao produto, e cada anel tem papéis próprios. Quem procura interessados sem um mapa encontra sempre os mesmos três.



O produto — o software em si. Aqui vivem os requisitos funcionais que a equipe escreve.

Nosso sistema — o produto mais quem o opera e o mantém: operador normal, suporte operacional, mantenedor. É o anel que a equipe mais esquece, e o que cobra a conta em produção.

O sistema contenedor — a organização em volta: quem se beneficia do resultado sem operar o software, quem comprou, quem patrocina. Aqui está o beneficiário funcional, o dono do processo.

O ambiente amplo — quem está fora da organização e ainda assim importa: regulador, auditoria, imprensa, o cidadão afetado, o concorrente, e o interessado negativo.

ALEXANDER, I. Understanding Project Sociology by Modeling Stakeholders. IEEE Software, v. 21, n. 1, p. 23-27, jan./fev. 2004.

Os papéis nomeados do modelo cebola

O valor do modelo não está no desenho: está na **lista de vagas a preencher**. Para cada papel abaixo, a pergunta é sempre a mesma — quem, com nome, ocupa isso no nosso projeto?

PAPEL	ANEL	QUEM É, NO SEU PROJETO
Operador normal	Nosso sistema	Quem opera o sistema como parte do trabalho diário
Suporte operacional	Nosso sistema	Quem ajuda o operador quando ele trava: mesa de ajuda, treinamento
Mantenedor	Nosso sistema	Quem corrige, atualiza e mantém o sistema no ar depois da entrega
Beneficiário funcional	Sistema contenedor	Quem recebe o resultado do trabalho do sistema sem operá-lo
Comprador e patrocinador	Sistema contenedor	Quem paga e quem defende politicamente a existência do projeto
Regulador	Ambiente amplo	Quem impõe exigência que não é negociável: lei, norma, auditoria
Beneficiário financeiro ou político	Ambiente amplo	Quem ganha com o sucesso sem participar: acionista, gestor público
Parte interessada negativa	Ambiente amplo	Quem perde algo se o sistema funcionar

Uma lista de papéis com vagas em branco é um resultado útil: ela mostra, antes de o projeto começar, de quem ninguém ainda falou.

A parte interessada negativa

Alguém que **perde algo se o sistema funcionar como planejado** — e que, por isso, tem interesse legítimo ou ilegítimo no fracasso dele.

Quem costuma ser

Quem **perde poder**: o setor cuja aprovação manual deixa de ser necessária

Quem **perde receita informal**: a fila que existia porque alguém a controlava

Quem **perde autonomia**: a equipe que tinha planilha própria e passa a seguir regra do sistema

Quem **fraudaria** o sistema: o atacante é uma parte interessada, com objetivos claros

O **concorrente**, quando o sistema é o produto da empresa

Por que ele entra no documento

Gera **requisito de segurança**: pensar como quem quer burlar é o que produz controle de acesso, trilha de auditoria e limite de tentativa

Gera **requisito de adoção**: sistema tecnicamente perfeito que ameaça alguém encontra resistência organizada e silenciosa

Gera **requisito de transição**: quando a perda é real e legítima, a resposta é negociação e plano, não ignorar

A **pergunta que revela essa pessoa** é simples e quase nunca é feita: *quem, dentro ou fora desta organização, fica em pior situação se este sistema der certo?* Se a resposta for "ninguém", a pergunta ainda não foi levada a sério.

Isso não é pessimismo nem teoria da conspiração: é a diferença entre um documento que descreve o mundo como ele é e um que descreve o mundo como seria conveniente que fosse.

Quem tem interesse em ver o sistema da sua equipe falhar?

Pense no projeto que a sua equipe escolheu. Não vale responder "ninguém". Procure o setor que perde uma aprovação, a pessoa que perde uma exceção, o processo informal que deixa de existir.

E a segunda pergunta: esse interesse é legítimo? Se for, ele não se resolve com senha e log — se resolve com negociação, e vira um requisito de transição no seu documento.

BLOCO 3

Priorizar os envolvidos

Identificar é metade do trabalho. A outra metade é decidir quanto peso cada um tem — e o que fazer quando dois interessados querem exatamente o oposto.

A matriz poder e interesse

Duas perguntas, e apenas duas, para cada nome da sua lista: **quanto poder** essa pessoa tem de alterar o rumo do projeto, e **quanto interesse** ela tem no resultado dele.



- Gerir de perto** — reunião frequente, participação nas decisões, validação direta. É onde o patrocinador e o dono do processo precisam estar.
- Manter satisfeito** — tem poder de derrubar o projeto, mas não acompanha o dia a dia. Informe no ritmo dele, com o nível de detalhe dele. Um diretor surpreendido é um projeto parado.
- Manter informado** — interesse alto, pouco poder formal. Costumam ser os usuários. Ignorá-los não derruba o projeto no papel; derruba na adoção.
- Apenas monitorar** — pouco de ambos. Revisite: poder e interesse mudam com o tempo, e quem estava aqui pode não estar mais.

A matriz é atribuída a MENDELLOW, A. Environmental Scanning: The Impact of the Stakeholder Concept (1981), e foi popularizada por EDEN, C.; ACKERMANN, F. Making Strategy (1998). O PMBOK a adota como ferramenta de análise de partes interessadas.

O que a matriz não resolve

A matriz é útil, é rápida e cabe num quadro branco. Também tem quatro limitações sérias, e usá-la sem saber disso é pior do que não usá-la.

01

É estática

Poder e interesse mudam. O gerente que não se importava passa a se importar quando o sistema afeta o indicador dele. Uma matriz feita uma vez, no começo, descreve um mundo que já não existe.

02

É subjetiva

Quem posiciona os nomes na matriz é você. Duas pessoas da mesma equipe produzem matrizes diferentes para o mesmo projeto — e nenhuma consegue justificar a diferença com dado.

03

Confunde poder com legitimidade

Quem grita mais alto aparece com poder alto. Isso não significa que a demanda dele seja legítima, nem que ignorar quem tem pouco poder seja defensável.

04

Trata a urgência como se não existisse

Uma exigência regulatória com prazo em 60 dias e uma preferência de longo prazo caem no mesmo quadrante. O modelo não tem onde registrar que uma delas não pode esperar.

Nada disso invalida a matriz. Significa que ela é **um ponto de partida que precisa de data, de dono e de revisão** — e que, para os casos difíceis, existe um modelo que responde exatamente ao que ela não responde.

O modelo de saliência: três atributos, não dois

Mitchell, Agle e Wood propõem avaliar cada parte interessada por três atributos independentes. A quantidade de atributos que ela reúne define o quanto ela deve pesar na sua atenção — o que os autores chamam de **saliência**.

ATRIBUTO 1

Poder (*power*)

Capacidade de impor a própria vontade sobre o projeto, seja por autoridade, por recurso ou por influência.

ATRIBUTO 2

Legitimidade (*legitimacy*)

A reivindicação é apropriada e reconhecida como válida: existe um direito real por trás dela, não só uma vontade.

ATRIBUTO 3

Urgência (*urgency*)

A demanda é sensível ao tempo e crítica para quem a faz. Esperar mais um trimestre não é uma opção aceitável.

QUANTOS ATRIBUTOS	COMO OS AUTORES CHAMAM	O QUE FAZER COM ELE
Um atributo	Latente: adormecido, discricionário ou exigente	Monitorar. Ainda não exige ação, mas pode ganhar outro atributo a qualquer momento
Dois atributos	Expectante: dominante, perigoso ou dependente	Atender de forma ativa. É onde mora a maior parte do trabalho real de negociação
Três atributos	Definitivo	Prioridade imediata e inegociável. Quem reúne os três decide o rumo do projeto

MITCHELL, R. K.; AGLE, B. R.; WOOD, D. J. Toward a Theory of Stakeholder Identification and Salience. Academy of Management Review, v. 22, n. 4, p. 853-886, 1997.

Os dois modelos aplicados ao mesmo caso

Um sistema de matrícula on-line em uma faculdade. Repare em como o segundo modelo muda a decisão em duas linhas.

PARTE INTERESSADA	PODER	INTERESSE	QUADRANTE DA MATRIZ	ATRIBUTOS DE SALIÊNCIA	O QUE MUDA
Diretoria	Alto	Baixo	Manter satisfeito	Poder + legitimidade	Nada muda: informe no ritmo dela
Secretaria acadêmica	Médio	Alto	Manter informado	Poder + legitimidade + urgência	Muda tudo: é definitiva, decide o rumo
Estudante	Baixo	Alto	Manter informado	Legitimidade + urgência	Dependente: precisa de alguém com poder para representá-la
Órgão regulador (MEC)	Alto	Baixo	Manter satisfeito	Poder + legitimidade + urgência	Muda tudo: exigência com prazo, não é negociável
Setor que perde a conferência manual	Baixo	Alto	Manter informado	Urgência apenas	Exigente: reclama alto, sem direito formal — mas indica risco de adoção

As duas linhas destacadas caíram em quadrantes diferentes na matriz e, na saliência, são as duas definitivas do projeto. Uma análise feita só com a matriz teria colocado o regulador em "manter satisfeito" — e descoberto o prazo dele tarde demais.

A conclusão prática: use a matriz para **enxergar o conjunto** rapidamente, e a saliência para **decidir os casos difíceis**, que são sempre os que envolvem prazo legal e quem não tem voz.

BLOCO 4

Quando ninguém perguntou a quem usa

Um programa público de saúde, nove anos de execução, 9,8 bilhões de libras, desmontado em 2011. Os requisitos existiam, estavam escritos, e foram construídos sem as pessoas que trabalhariam com o sistema.

National Programme for IT — Serviço de saúde do Reino Unido

Lançado em 2002 para criar um prontuário eletrônico único para todo o serviço público de saúde inglês. Foi, à época, o maior projeto civil de tecnologia da informação do mundo.

2002

ano de lançamento do programa, com prontuário único para todo o serviço de saúde

£ 7,3 bi

já haviam sido gastos até março de 2012, segundo a declaração oficial de benefícios

£ 9,8 bi

é a estimativa de custo final do programa reportada ao Parlamento

2011

ano em que o governo anunciou o desmantelamento do programa

O diagnóstico do próprio Parlamento britânico, na comissão que investigou o caso:

"A ideia de uma padronização implacável provou-se ilusória. Gerir uma mudança dessa natureza de cima, centralmente, simplesmente não é possível."

Comissão de Contas Públicas da Câmara dos Comuns, relatório sobre o programa desmantelado

Antes da análise, uma pergunta para a turma: os requisitos desse programa foram escritos por gente incompetente, ou por gente competente que falou com as pessoas erradas?

Câmara dos Comuns do Reino Unido, Comissão de Contas Públicas: The dismantled National Programme for IT in the NHS (HC 294, 2013).

O caso lido com os modelos desta aula

O QUE ACONTECEU	COMO O MODELO DE HOJE EXPLICA
O programa foi desenhado centralmente e imposto aos hospitais	Os requisitos foram levantados no anel do sistema contenedor — gestores e ministério — e quase nada no anel de "nosso sistema", onde estão os operadores normais: médicos e enfermeiros
Cada hospital trabalhava de um jeito diferente	Os requisitos de parte interessada foram tratados como se houvesse uma única classe de usuário para todo o país, quando havia centenas de classes com processos distintos
Os clínicos resistiram à adoção	Profissionais com legitimidade e urgência altas, e poder formal baixo, caem em "manter informado" na matriz. Foram informados. Informar não é envolver — e sem eles não havia validação possível
O sistema entregue não era o sistema necessário	Verificação passou: o software fazia o que a especificação dizia. Validação falhou: a especificação não descrevia a necessidade real de quem usaria

Nenhuma linha acima é um erro de programação. Todas são consequência de **com quem se falou** — e de ter confundido informar com envolver.

Vale a ressalva honesta: um programa dessa escala falha por muitas razões ao mesmo tempo — contrato, tecnologia, política e prazo. Requisitos e partes interessadas são uma dessas razões, das mais citadas, e não a explicação completa.

O que fica de hoje

PONTO	REGRA PRÁTICA
O processo	Cinco atividades: elicitção, análise, especificação, validação e gerência. As quatro primeiras produzem o requisito; a quinta o mantém honesto.
Espiral, não fila	Especificar levanta perguntas, validar devolve trabalho, e o mundo muda durante o projeto. Voltar não é retrabalho: é o processo funcionando.
Três níveis	Parte interessada, sistema e software. Requisito de software que não sobe até a necessidade de alguém com nome foi inventado pela equipe.
Parte interessada	Quem afeta ou é afetado. Inclui quem mantém, quem regula, quem é afetado sem usar — e quem prefere que o sistema falhe.
Identificar	Modelo cebola: quatro anéis, papéis nomeados. Vaga em branco na lista é a pergunta que ainda não foi feita.
Priorizar	Matriz poder e interesse para enxergar o conjunto; saliência (poder, legitimidade, urgência) para decidir os casos difíceis.
A lição do caso	Verificação passa e validação falha quando se fala com as pessoas erradas. Informar não é envolver.

Escrever bem um requisito é metade da competência. A outra metade é **saber de quem ele precisa vir** — e conseguir defender essa escolha quando dois interessados querem o oposto.

Glossário: o termo em inglês e o que ele quer dizer

A bibliografia da área e as normas internacionais estão em inglês. Guardar a tradução ao lado do termo evita decorar palavra sem conceito.

<code>requirements engineering</code> engenharia de requisitos Processo de descobrir, analisar, documentar e verificar requisitos
<code>stakeholder</code> parte interessada Quem afeta ou é afetado pelo sistema
<code>elicitation</code> elicitação, levantamento Descobrir a necessidade junto com os envolvidos
<code>specification</code> especificação Escrever o requisito de forma verificável
<code>validation</code> validação Confirmar que é o sistema certo, com quem pediu
<code>verification</code> verificação Confirmar que foi construído do jeito certo
<code>requirements management</code> gerência de requisitos Controlar mudança, versão e rastreabilidade
<code>baseline</code> linha de base Versão aprovada que só muda por processo formal

<code>traceability</code> rastreabilidade Ligação entre necessidade, requisito e teste
<code>StRS</code> requisitos de parte interessada Nível do interessado, na linguagem dele
<code>SyRS</code> requisitos de sistema Nível do sistema: software, hardware, pessoas
<code>SRS</code> requisitos de software Nível da equipe que vai construir
<code>functional requirement</code> requisito funcional O que o sistema deve fazer
<code>non-functional requirement</code> requisito não funcional Sob que qualidade ele deve fazer
<code>business rule</code> regra de negócio Política da organização, existe sem o sistema
<code>user class</code> classe de usuário Grupo de usuários com necessidades semelhantes

<code>product champion</code> campeão do produto Representante nomeado de uma classe de usuário
<code>sponsor</code> patrocinador Quem dá autoridade, orçamento e define o sucesso
<code>domain expert</code> especialista de domínio Quem conhece a regra que ninguém escreveu
<code>onion model</code> modelo cebola Mapa em anéis para identificar interessados
<code>negative stakeholder</code> parte interessada negativa Quem perde algo se o sistema funcionar
<code>power / interest</code> poder / interesse Os dois eixos da matriz de priorização
<code>salience</code> saliência Peso do interessado por poder, legitimidade e urgência
<code>scope creep</code> inchaço de escopo Crescimento não controlado do que foi combinado

De onde veio cada afirmação desta aula

O PROCESSO E A NORMA

SOMMERVILLE, I. **Engenharia de Software**. 10. ed. Pearson — capítulo 4, o processo de requisitos.

processo em espiral e atividades

KOTONYA, G.; SOMMERVILLE, I. **Requirements Engineering: Processes and Techniques**. Wiley, 1998.

definição do processo

ISO/IEC/IEEE 29148:2018 — Requirements engineering.

os três níveis: StRS, SyRS e SRS

WIEGERS, K.; BEATTY, J. **Software Requirements**. 3. ed. Microsoft Press, 2013.

classes de usuário e campeão do produto

BOEHM, B. **Software Engineering Economics**. Prentice-Hall, 1981.

custo de corrigir por fase (Aula 1)

PARTES INTERESSADAS E O CASO

FREEMAN, R. E. **Strategic Management: A Stakeholder Approach**. Pitman, 1984.

a definição clássica de parte interessada

ALEXANDER, I. **Understanding Project Sociology by Modeling Stakeholders**. IEEE Software, 21(1), 2004.

o modelo cebola

MITCHELL; AGLE; WOOD. **Toward a Theory of Stakeholder Identification and Salience**. AMR, 22(4), 1997.

o modelo de saliência

MENDELOW, A. (1981) · EDEN; ACKERMANN. **Making Strategy** (1998).

a matriz poder e interesse

UK Public Accounts Committee. The dismantled National Programme for IT in the NHS. HC 294, 2013.

o caso do Bloco 4

FERNANDES, J. M.; MACHADO, R. J. **Requisitos em projetos de software e de sistemas de informação**. Novatec, 2018.

bibliografia básica da disciplina

Técnicas de elicitación

Agora que sabemos **com quem** falar, a próxima aula trata de **como** falar: entrevista e questionário, perguntas abertas e fechadas, os vieses de quem entrevista, e o que fazer quando o usuário não consegue descrever o próprio trabalho — observação, oficina e prototipação.

Para chegar pronto no dia 10

- Ter o cliente da equipe definido, com nome e forma de contato
- Trazer uma primeira lista de partes interessadas desse cliente, com os papéis do modelo cebola
- Marcar, na lista, quem sua equipe ainda não consegue nomear — a vaga em branco é o resultado mais útil
- Levar o REQUISITOS.md da Aula 2, porque as reescritas vão virar roteiro de entrevista